D

Dumbhanengue ERP

Dumbhanengue ERP — Checklist de préparation à la production (français)

Version 1.1.1+3 · 16 août 2026

Dumbhanengue ERP

Table des matières

1. Intégrité des données & chemin de l'argent (CRITIQUE) 3
2. Sécurité 4
3. Architecture & qualité de code 5
4. Performance & passage à l'échelle 6
5. Business / SaaS 6
6. Tests 7
7. Opérations de production 8
8. Dépendances 10
9. Docs & ménage 10
Priorités immédiates (prochaine session de travail) 11

> Origine : audit de production E2E (4 août 2026). Faits de base : flutter analyze propre · 381 tests verts (7 août) · Flutter 3.44.6 / Dart 3.12.2.
> Légende des statuts : fait (fusionné/vérifié) · en cours · en file · bloqué/besoin d'une ressource externe · à vérifier hors du dépôt.
>
> Source faisant autorité (portugais) : `../pt/PRODUCTION_CHECKLIST.md` · détails de l'audit : 15-audit-status.md · runbooks de migration : W4+H2 16-migracao-w4-h2.md · W11 (lots FIFO + ventes au poids) 18-migracao-w11.md

---

1. Intégrité des données & chemin de l'argent (CRITIQUE)

#	Élément	Statut	Notes
1.1	Traitement atomique des ventes sur Supabase (RPC process_sale + SupabaseSaleProcessor)		Transaction DB unique : vente+articles, verrou/décrément du stock, écriture de caisse. Schéma + câblage + tests. E2E live 22/22 PASS (5 août) + sql_check.sh exécute maintenant le RPC (6.2).
1.1b	Traitement atomique des achats sur Supabase (RPC process_purchase + SupabasePurchaseProcessor)		Reproduit le chemin des ventes ; l'aide partagée lock_products_or_raise garde les deux RPC synchronisés.
1.1c	Enregistrement atomique des dettes sur Supabase (RPC register_debt)		Dette + entrée en attente liée dans une seule transaction DB (le chemin composite conserve une sémantique volontairement laxiste sur SQLite). E2E live 22/22 PASS (5 août) : register_debt + settle_debt sur le projet live.
1.1c2	Règlement atomique des dettes sur Supabase (RPC settle_debt)		Audit C4 : RPC settle_debt (SECURITY DEFINER, idempotent, scopé magasin/locataire) + délégation de SettleDebtUseCase via IDebtAtomicProcessor.settle. Schéma appliqué au projet live (7.8).
1.1d	Idempotence dans les RPC (sûr en cas de réessai)		process_sale/process_purchase/register_debt sortent tôt quand l'id existe déjà (id = clé d'idempotence).
1.2	Rôle super-admin durci (table DB/claims, pas une chaîne d'email)		Audit C3 : table public.super_admins + RPC is_super_admin() SECURITY DEFINER (schema.sql) + contrôle RPC côté client (isSuperAdminProvider). Schéma appliqué au projet live (7.8) ; vérifier que la confirmation d'email est ACTIVÉE dans le dashboard (2.1).
1.3	Accès des employés aux données métier (RLS par store_employees)		RLS scopé magasin is_my_store() appliqué à toutes les tables métier (C2, 5 août) — les employés voient/écrivent les données de leur propre magasin ; le propriétaire couvre tout.

1.4	Application des limites de plan en DB (triggers sur stores / store_employees)		enforce_store_limit / enforce_employee_limit ajoutés à schema.sql ; appliqués au projet live (7.8).
1.4b	Matrice de niveaux d'accès (6 août) : les achats sont de niveau MANAGER — le propriétaire ou le manager enregistre/consulte les achats ; un employé ordinaire ne peut pas (même pas via PostgREST direct)		Aide is_manager_of_store + garde process_purchase + politiques RLS purchases/purchase_items dans schema.sql et supabase/upgrade.sql (Partie 4) ; l'UI masque Achats/Rapports (employé) et Magasins (non-propriétaire) via currentAccessLevelProvider ; régression [8/8] dans sql_check.sh (employé bloqué, manager autorisé).
1.4c	Activation des employés + création de produits (7 août) : le propriétaire active/désactive les employés (un employé désactivé perd immédiatement l'accès, app et serveur) ; un employé ordinaire ne crée PAS de produits — il vend seulement		store_employees.is_active (DEFAULT TRUE) + RPC set_employee_active (propriétaire uniquement) + is_my_store/is_manager_of_store exigeant un lien ACTIF — schema.sql et supabase/upgrade.sql Partie 19 ; AuthGate affiche « Access suspended » pour les comptes désactivés ; le FAB « Nouveau produit » est masqué quand currentAccessLevelProvider == employee (propriétaire/gérant le gardent) ; section sql_check.sh [6b] (désactivation → is_my_store FALSE, réactivation → TRUE, non-propriétaire rejeté) ; tests 381/381.
1.5	Décrément atomique du stock sur Supabase (pas de perte de mise à jour)		Résolu à l'intérieur des RPC process_sale/ process_purchase (FOR UPDATE + UPDATE gardé).
1.7	Argent en centimes entiers (remplacer double/REAL)		W4 (6 août — dernière session) : migration complète vers les centimes entiers — domaine (*Cents), SQLite (INTEGER + migration ×100 idempotente par affinité de colonne), Supabase (BIGINT + migration ×100 gardée — Partie 7 de supabase/upgrade.sql), RPC comparant en centimes (pas de ROUND(x*100)), catalogue des plans en centimes, UI (Money.parse/ Formatters.currency(cents)). 381/381 tests verts.
1.10	IDs sans collision		AppIds.newId() (horodatage + entropie) remplace tous les IDs microsecondsSinceEpoch.
1.16	SettleDebtUseCase dans IUnitOfWork (atomicité SQLite)		Le règlement de la dette et l'écriture de caisse se replient ensemble sur SQLite.
1.17	Critique (5 août) : bug de précédence d'opérateur dans les RPC atomiques — `'t'		p->>'id' se parse en ('t'
1.18	6 août : settle_debt estampille la DATE DE RÉALISATION (date = now()) sur l'entrée en attente liée — même sémantique que le markAsPaid local (SQLite). Avant, le mode SaaS gardait la date d'ENREGISTREMENT de l'entrée et le cash flow reconnaissait l'encaissement dans la mauvaise période		schema.sql + supabase/upgrade.sql (Partie 8.3, idempotent) + assertion de date dans sql_check.sh.

2. Sécurité

#	Élément	Statut	Notes
2.1	Vérifier que la confirmation d'email est activée dans Supabase		Contrôle Dashboard requis.
2.2	Activer la 2FA sur le compte super-admin		Dashboard / fonctionnalité Supabase.
2.3	Journal d'audit pour les décisions admin (approbation/rejet/modifs de plan)		H8 (6 août) : audit_log complet — DDL + triggers trg_audit_* (tenants/plan_requests/plans) + get_audit_log dans supabase/upgrade.sql (Partie 5) + decided_by (S4, 5 août). Cohérent avec AUDIT_REPORT H8 .
2.4	Retirer les défauts dev de l'URL Supabase / clé anon ; fail-fast en release		Fail-fast en release : SupabaseBootstrap lève une erreur si les défauts SUPABASE_URL/SUPABASE_ANON_KEY sont embarqués dans une release NON-prod (S2, 5 août ; ajusté le 9 août — le déploiement est un SEUL projet Supabase dotklbpqmlnhcdifiukd partagé entre dev/prod ; une release prod avec les propres identifiants du projet est légitime ; main.dart bloque toujours les releases sans --flavor prod). Flavors dev/prod implémentés (7.2, 5 août).
2.5	Durcissement du login : email strict, mot de passe min-8, champ de confirmation, bascule de visibilité		login_screen.dart. Mot de passe EMPLOYÉ aligné sur le minimum de 8 caractères (M7, 5 août).
2.6	AuthGate : état d'erreur explicite du flux d'authentification (pas de drop silencieux du login)		auth_gate.dart.
2.7	CSP web + métadonnées quand le web sort		Métadonnées (5 août). La CSP reste (uniquement si le web est une cible de release).
2.8	Signalement de crash (Sentry/Crashlytics)		Sentry intégré (crash_reporting.dart) ; DSN via --dart-define=SENTRY_DSN. Mis à jour vers sentry_flutter 9.26.0 (5 août) — 8.14.2 cassait le build de release (Kotlin 1.6 vs 2.x intégré).
2.9	Session d'authentification dans un stockage SÉCURISÉ (Keychain/Keystore), pas shared_preferences		S1 (6 août) : SecureSupabaseLocalStorage (session) + SecureGotrueAsyncStorage (vérificateur de code PKCE) + authOptions dans SupabaseBootstrap (natif). Le défaut du SDK écrivait les tokens access/refresh en XML en clair. Le web garde le défaut (flutter_secure_storage 10.x ne supporte pas le web — limitation documentée). Installations existantes : 1 déconnexion après le changement (la session migre vers Keychain/Keystore).

3. Architecture & qualité de code

#	Élément	Statut	Notes
3.1	Supprimer la dépendance morte dio + enregistrement		pubspec.yaml + service_locator.dart.

3.2	Supprimer la hiérarchie inutilisée Failure		lib/core/errors/failure.dart supprimé (les exceptions du domaine restent).
3.3	Renommer les contrats *LocalDataSource (trompeurs pour les implémentations Supabase)		Cosmétique ; les docs le reconnaissent. Priorité basse.
3.4	Recherche de produit par map dans les feuilles vente/achat		new_sale_sheet.dart, add_purchase_sheet.dart.

4. Performance & passage à l'échelle

#	Élément	Statut	Notes
4.1	Pagination (.range/.limit) sur tous les getAll Supabase		H2 (6 août — dernière session) : pagination keyset sur le getAll des 6 datasources distants (products, sales, cash, credit, purchases, customers) — l'aide fetchAllKeyset (lots de pageSize clés par le curseur stable id, gt(id, last) à la page suivante) remplace le .limit(1000) unique qui tronquait les magasins de 10k+ lignes ; l'ordre d'affichage est préservé (les produits sont re-triés par nom dans le datasource ; les autres dans les providers).
4.2	RPC d'agrégation côté serveur pour KPIs/rapports/cash flow		W5 (6 août) : RPC get_dashboard_kpis + provider avec fallback local + migration 009. H2 (6 août — dernière session) : nouveau get_cash_flow (entrées PAYÉES de la période) et get_sales_report (chiffre d'affaires, bénéfice brut, top 5 vendeurs et ventes par jour dans le fuseau de l'appareil) — SECURITY DEFINER avec garde is_my_store, valeurs BIGINT (centimes). GetCashFlowUseCase/GetSalesReportUseCase ont gagné un serverRepository optionnel (SupabaseCashFlowRepository/ SupabaseSalesReportRepository, enregistrés seulement en mode SaaS) ; local/tests gardent l'agrégation côté client. Consolidé dans supabase/upgrade.sql (Parties 8.6–8.7) + étape [11/11] dans sql_check.sh.
4.3	Recherches ciblées de SettleDebtUseCase (WHERE id/debt_id)		Supabase délègue maintenant au RPC ciblé settle_debt (C4) ; le chemin composite (SQLite/mémoire) reste O(toutes les dettes + toutes les transactions).
4.4	Différer l'ouverture de SQLite hors de l'isolat principal		Frame de démarrage.
4.5	Cache hors-ligne + file de synchronisation (local-first)		Stratégique ; plus gros gain UX pour le marché cible.

5. Business / SaaS

#	Élément	Statut	Notes
5.1	Rejeter le downgrade de plan sous l'usage actuel (trigger DB)		Prolonge le travail des triggers du §1.4.

5.2	Intégration paiements/facturation		Service externe (Stripe/Mercado Pago).
5.3	Upgrade/downgrade en libre-service		Dépend de 5.2.
5.4	Flux de suppression/annulation/édition (produits, clients, ventes), ajustements de stock		Les commerçants en ont besoin au quotidien.

6. Tests

#	Élément	Statut	Notes
6.1	Tests de contrat des datasources Supabase		Le chemin de l'argent a une couverture automatisée via `sql_check.sh` (16/16, incl. W11 : lots FIFO, achats créant des lots et l'invariant `stock = Σ lots`), qui EXÉCUTE process_sale/register_debt/settle_debt/get_dashboard_kpis/ get_cash_flow/get_sales_report en tant qu'app_user et vérifie les effets + les valeurs. Les tests de contrat unitaires Dart pour les datasources distants restent ().
6.2	Suite de tests SQL/RLS (Docker Postgres + sqlcheck_stub.sql)		Audit H3 : scripts/sql_check.sh (rôle non-superuser app_user pour que la RLS s'applique réellement ; apply du schéma + super admin + isolation). Re-validé le 5 août après C3/C4. Étendu avec la section [4/4] (5 août) : EXÉCUTE `process_sale`/`register_debt`/`settle_debt` en tant qu'`app_user` et vérifie les effets (vente+article+stock+caisse, dette+entrée en attente, règlement de la MÊME entrée en attente, stock négatif) — régression du bug 1.17 (22P02). 6 août : assertion de DATE DE RÉALISATION après settle_debt (1.18) + étape [10/10] get_dashboard_kpis (baseline + delta après process_sale). Dernière session (6 août) : étapes [1b] (upgrade.sql appliqué 2× — idempotence), [11/11] (get_cash_flow/get_sales_report — H2) et [12/12] (migration W4 NUMERIC→centimes avec de vraies données 25.50→2550, sans double application). Exécuté en CI (jobs sql-check de ci.yml + release.yml, Postgres 16, scram sur TCP) — 12/12 verts. 9 août (W11) : étendu avec la diminution des lots FIFO dans les ventes (péremption la plus proche d'abord, surplus au lot suivant), les achats créant des lots (avec péremption optionnelle) et l'invariant `stock = Σ lots` — 16/16 verts.
6.3	Tests de widgets : dashboard, clients, rapports, achats, admin, onboarding, auth_gate		auth_gate_test.dart — 5 tests : rôle admin (DB) → AdminShell (incl. locataire en attente), pas de rôle → flux locataire, échec transitoire fail-open, déconnecté → Login. Les autres écrans suivent dans le §7 de l'audit.

6.4	Tests d'intégration (integration_test/)		integration_test/access_levels_web_test.dart (E2E navigateur du contrôle d'accès propriétaire/gérant/employé contre le vrai Supabase, via test_driver/) — lancé avec flutter drive --no-dds --driver=test_driver/integration_test.dart --target=integration_test/access_levels_web_test.dart -d chrome --dart-define-from-file=env/dev.env + définitions TEST_* (chromedriver sur le port 4444 ; --no-dds contourne le bug AppConnectionException de dwds sur Windows). Re-validé le 6 août après la migration W4/H2 : PASS (propriétaire → tout ; gérant → achats/rapports sans magasins ; employé → opérations uniquement). + `integration_test/cash_closing_summary_live_test.dart` (E2E du payload RÉEL du RPC get_cash_closing_summary contre Supabase : capture le payload exact {p_store_id, p_day} avec le jour local en UTC via une couture d'observation et le transmet au vrai serveur — prouve l'acceptation, l'agrégation cohérente côté serveur (total = Σ par méthode ; attendu = caisse uniquement), la chaîne enregistrée de l'app et la garde is_my_store) — lancé avec flutter drive --no-dds --driver=test_driver/integration_test.dart --target=integration_test/cash_closing_summary_live_test.dart -d chrome --dart-define-from-file=env/dev.env + --dart-define=TEST_OWNER_EMAIL=... --dart-define=TEST_OWNER_PASSWORD=....
6.5	Seuil de couverture en CI		Garde active (5 août) : flutter test --coverage échoue sous 45% en CI.
6.6	Nouveaux tests de délégation pour les processeurs atomiques (vente/achat/dette)		process_sale_use_case_test.dart, add_purchase_use_case_test.dart, add_debt_use_case_test.dart.
6.7	Test unitaire pour les AppIds sans collision		test/core/utils/ids_test.dart.
6.8	Accessibilité : test d'échelle de texte 2.0 + constantes UI nommées		W8 (6 août) : test/presentation/screens/accessibility_text_scale_test.dart (login utilisable à l'échelle de police max) + lowStockThreshold sur le dashboard (M8).

7. Opérations de production

#	Élément	Statut	Notes
7.1	Vraie config de signature Android (pas de clés debug)		Keystore android/app/upload-keystore.jks généré (alias upload, RSA 2048, valide jusqu'en 2053) + android/key.properties rempli (gitignoré ; PKCS12 = un seul mot de passe store/clé). Docs : key.properties.example, guide développeur (pt/en). MANQUANT (externe) : publier le 1er AAB, activer Play App Signing et enregistrer le SHA-256 de la clé upload dans la console.

7.2	Flavors dev/prod + stratégie dart-define		Flavor dev (applicationIdSuffix .dev, label « Dumbhanengue » — pas de suffixe visible dans le nom) et prod dans build.gradle.kts ; définitions obligatoires via --dart-define-from-file (env/dev.env, env/prod.env.example) ; gardes de release : main.dart exige le flavor prod (FLUTTER_APP_FLAVOR est injecté par le framework depuis --flavor — définition réservée), SupabaseBootstrap rejette les défauts dev embarqués (S2), et Gradle échoue prodRelease sans keystore (5 août).
7.3	CI/CD (GitHub Actions : analyze → test → build)		.github/workflows/ci.yml — analyze, test+coverage, build APK de release (flavor dev + env/dev.env), sql-check (service Postgres). Release : .github/workflows/release.yml (tag v* → Fastlane : AAB prod signé → upload Internal testing, 5 août).
7.4	Icônes d'app, splash, branding, métadonnées store		Icônes (W7, 6 août) : Android adaptatif (mipmap-anydpi-v26 + drawables), iOS complet (21 images), web (favicon + manifest) ; sources dans assets/branding/ + générateur tool/generate_icons_test.dart ; défauts du template supprimés. Redesign v3 (8 août) : image source officielle sombre « sac + D » (assets/branding/dumbhanenge_logo.png) — générateur réécrit pour la traiter (legacy/fg 62%/mono par luminance chrome) + adaptive_icon_background sombre #19191A ; natifs régénérés. Splash sombre (8 août) : in-app (_SplashScreen + splash_logo.png embarqué), Android natif (launch_background + values-v31 pour Android 12+) et iOS (storyboard + LaunchImage) ; splash de chargement sombre web dans index.html. Icône officielle (v4.2, 8 août) : source assets/branding/icon_official.png — design fourni par l'utilisateur : sac 3D bleu (dégradé bleu ciel→indigo) avec « D » blanc et œillets, fond blanc ; keying flood-fill des bords (préserve le « D »/les œillets enclos, supprime fond+ombre) + adaptive_icon_background blanc #FFFFFF ; mono par luminance 60 ; natifs régénérés. Métadonnées store (consoles) restent externes .
7.5	Politique de confidentialité / CGU		Légal.
7.6	Sauvegarde/export des données		Export CSV + sauvegardes Supabase. Flux de sauvegarde DB exercé le 6 août (pg_dump via pooler 5432 → backup_pre_w4.sql, gitignoré) avant la migration W4/H2 (7.8b).
7.7	Budget de taille du build de release (obfuscation, split-per-abi)		Ajouter aux scripts de build.
7.8	schema.sql appliqué au Supabase live + vérification lecture seule		5 août 2026 : appliqué en UNE transaction (COMMIT) via le pooler aws-0-eu-north-1 (projet dotklbpqmlnhcdifiukd — l'hôte direct est IPv6 uniquement, utiliser le pooler). Constat pré-application : seulement les 13 tables de base — la couche de durcissement C3/C4 n'avait JAMAIS été appliquée (ancien is_super_admin basé sur l'email ; pas de settle_debt, pas de RPC atomiques, pas de triggers de limite). Post-application : 14 tables · 52 politiques · 39 index · 20 triggers · 17 fonctions · plans intacts (3, sans doublon). Re-vérification lecture seule 10/10 PASS (scripts/sql_check_live.sh).

7.8b	`upgrade.sql` appliqué en live (W4 centimes + RPC H2) + vérification lecture seule		6 août 2026 : appliqué en UNE transaction (-1) via le pooler aws-0-eu-north-1 (PostgreSQL 17.6) : 9 colonnes monétaires → BIGINT (centimes), zéro valeur fractionnaire, RPC H2 (get_cash_flow/ get_sales_report/get_dashboard_kpis) actifs ; sql_check_live.sh 14/14 PASS ; idempotence confirmée (réapplication sans effet). Sauvegarde backup_pre_w4.sql (gitignorée). Runbook : [`docs/fr/16-migracao-w4-h2.md`](16-migracao-w4-h2.md) (chapitre 16).
7.8c	`upgrade.sql` Partie 22 appliquée en live (W11 : lots FIFO + ventes au poids, quantités ×1000) + vérification lecture seule		En attente : appliquer la Partie 22 de supabase/upgrade.sql (migration millièmes + backfill des lots + RPC FIFO) en UNE transaction, puis sql_check_live.sh (maintenant 31 contrôles, incl. W11 25–31) + publier l'app W11 dans la même fenêtre (pas de clients hérités écrivant des échelles mixtes). Invariant post-migration : stock = Σ lots actifs (contrôle 31). Runbook : [`docs/pt/16-migracao-w11.md`](../pt/16-migracao-w11.md) (cap 16) / 18-migracao-w11.md.
7.9	Pipeline iOS (TestFlight)		W1 (6 août) : ios/Podfile + lane ios_testflight dans le Fastfile + .github/workflows/ios.yml (runner macOS : analyze → test → build+sign iOS → TestFlight ; secrets ASC_KEY_ID/ASC_ISSUER_ID/ASC_KEY_BASE64). Manquant externe : compte Apple + certificats dans les secrets du dépôt.
7.10	Guide de go-live (ops)		W3 (6 août) : docs/{pt,en}/14-go-live-ops.md — secrets CI (KEYSTORE_BASE64 + mots de passe, PLAY_*), export du keystore en base64, checklist Play Console (Play App Signing, 2FA, confirmation d'email), étapes TestFlight.

8. Dépendances

#	Élément	Statut	Notes
8.1	Supprimer dio		Dépendance morte supprimée.
8.2	Migrer sqlite3 2.9.0 → 3.x + sqlite3_android		W10 (6 août — dernière session) : sqlite3 2.9.0 → ^3.5.1 — depuis 3.0.0 le package utilise des build hooks (native assets) et compile/embarque SQLite pour Android/iOS/desktop ; supprimé sqlite3_flutter_libs (EOL) et le loader manuel (open.overrideFor/ DynamicLibrary), dispose() → close(). Pas besoin de sqlite3_android (le hook couvre Android).
8.3	dart pub upgrade + re-vérifier 175 tests		6 août (dernière session) : après W10, dart pub get + suite complète 300/300 tests verts + flutter analyze 0 issues. 7 août : suite 381/381 (activation/désactivation des employés + création de produits propriétaire/gérant).

9. Docs & ménage

#	Élément	Statut	Notes
9.1	Mettre à jour README/docs pour la suppression de dio/Failure et le nouveau chemin RPC		Docs synchronisés dans les audits du 5/6 août (en/pt) : architecture sans Failure, RPC atomiques, KPIs côté serveur, flavors, pipelines Android+iOS, guide 14 (go-live).

9.2	CHANGELOG + processus de release		CHANGELOG.md créé (6 août) avec l'historique complet (audits des 4/5/6 août, fonctionnalités, correctifs).

Priorités immédiates (prochaine session de travail)

1. Externe (bloqueurs de store) : activer Play App Signing + enregistrer le SHA-256 de la clé upload (7.1) ; vérifier la confirmation d'email ACTIVÉE + 2FA dans le dashboard Supabase (2.1/2.2) ; comptes Google Play/Apple + secrets CI (7.9) ; politique de confidentialité/CGU (7.5).
2. W2 — facturation (différé par décision produit ; les limites sont déjà appliquées côté serveur).
3. Appliquer `supabase/upgrade.sql` au Supabase live + `sql_check_live.sh` 14/14 — FAIT le 6 août (pooler eu-north-1, transaction unique -1, PostgreSQL 17.6) : 9 colonnes monétaires → bigint, zéro valeur fractionnaire, 14/14 PASS, idempotence confirmée ; sauvegarde backup_pre_w4.sql (gitignorée). Runbook : docs/fr/16-migracao-w4-h2.md. *Tous les éléments internes (W4, H2, W10, consolidation SQL, application live) sont faits — ne restent que les externes (1) et W2 (2).*